

The Production Cost Playbook for Agentic RAG and LLM Systems

A modern field guide to diagnosing runaway spend, redesigning architecture, and reducing cost without wrecking quality

What this guide solves. Why production LLM systems become expensive, how to diagnose each cost source, which interventions usually save the most, how to evaluate trade-offs, and how to implement routing, retrieval funnels, caching, compaction, and escalation with code patterns.

Audience	Use case	Length	Design note
Platform engineers, ML engineers, architects, GenAI product leads	Agentic workflows, RAG assistants, internal copilots, transactional bots	Long-form professional handbook	Charts are illustrative and intended for operating decisions

Contents

- 1. Executive summary
- 2. 1. Why LLM cost becomes painful in production
- 3. 2. The seven major sources of high cost
- 4. 3. Build a cost baseline before changing architecture
- 5. 4. Cut orchestration before you cut model quality
- 6. 5. Context is usually more expensive than you think
- 7. 6. RAG cost anatomy: why retrieval-heavy systems bloat
- 8. 7. Use routing to protect premium models
- 9. 8. Cache the right things, not only the final answer
- 10. 9. Precompute work outside the request path
- 11. 10. Failure loops and retries are hidden cost multipliers
- 12. 11. Observability: the control room for cost reduction
- 13. 12. Trade-offs: where aggressive optimization can backfire
- 14. 13. Reference architecture for a low-cost production assistant
- 15. 14. Engineering checklist by maturity stage
- 16. 15. Code patterns that reduce cost in practice
- 17. 16. Detailed playbook: reason, symptom, fix, trade-off
- 18. 17. Cost metrics every production team should track
- 19. 18. Common anti-patterns and what to do instead

- 20. 19. Example implementation patterns
- 21. 20. Final recommendations
- 22. 21. Playbook for enterprise knowledge assistants
- 23. 22. Playbook for coding assistants and developer agents
- 24. 23. Playbook for workflow and transactional agents
- 25. 24. Chunking and corpus design in depth
- 26. 25. Retrieval evaluation in depth
- 27. 26. Memory design in depth
- 28. 27. Governance and FinOps for LLM systems
- 29. 28. Migration plan: how to optimize without breaking production
- 30. 29. Case-study style scenarios
- 31. 30. What good looks like
- 32. Appendix A. Annotated code walkthroughs

1. Executive summary

Most teams assume their LLM bill is high because their primary model is expensive. In production, that is rarely the full story. Cost usually balloons because the architecture allows too many model invocations, too many tokens per invocation, too much irrelevant retrieval context, too much conversation replay, and too many retries caused by weak grounding. The result is a system that looks sophisticated in demos but becomes painful to operate at scale.

This guide treats cost reduction as a systems problem rather than a single-model pricing problem. It explains where spend actually accumulates, how to instrument a cost baseline, which levers usually produce the largest savings, and what trade-offs appear when you push too aggressively on efficiency. The emphasis is practical: architecture patterns, engineering checklists, metrics, code snippets, operating rules, and design decisions a production team can adopt immediately.

A simple framing helps. There are only five durable ways to lower the total cost of an LLM application: reduce the number of calls, reduce the number of tokens, reduce the price of the model used for each task, increase the share of requests answered from cache or precomputation, and reduce failure loops that force retries. Everything else is a variant of one of these five.

In agentic systems, step count is often the dominant hidden tax. In RAG systems, context bloat is the most common silent killer. In both, weak routing causes premium models to be used for cheap tasks, while weak observability prevents teams from realizing which pattern is driving spend. Fix those issues first and the economics of the system usually change dramatically.

Throughout this guide, figures are illustrative rather than vendor specific. That is intentional: the exact tariffs of any provider change over time, but the structural reasons for high cost remain stable. A production leader should therefore optimize around architecture, observability, and task allocation, not only around a temporary provider price sheet.

2. 1. Why LLM cost becomes painful in production

The path from proof of concept to production usually changes the economics of an LLM system in three ways. First, traffic increases and exposes every inefficient design choice. Second, product requirements become stricter: auditability, freshness, latency targets, permissions, fallback flows, and multi-turn sessions all add complexity. Third, teams introduce more orchestration in an attempt to improve quality, and each layer quietly adds cost.

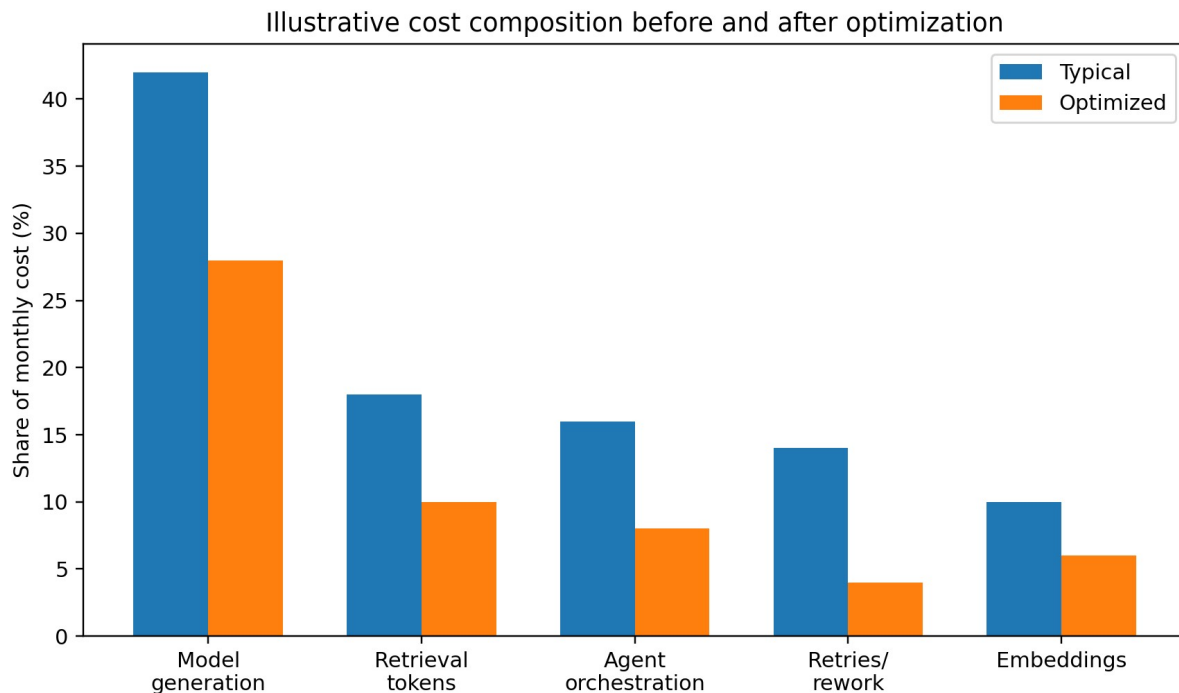
A proof of concept can survive a monolithic prompt, a premium model, and an oversized context window because request volume is low. A production assistant cannot. Once thousands of requests begin to hit the service, the architecture behaves like a multiplier. If every user question results in five LLM calls instead of two, or twenty retrieved chunks instead of four, the bill does not grow linearly with user value; it grows with inefficiency.

Another reason cost gets out of control is that quality issues masquerade as cost issues. A system that retrieves poorly, hallucinates, or misses structured data often triggers retries, human

escalations, or secondary agent passes. Leaders then blame model pricing, while the real problem is that the first answer path was not grounded enough. A high quality first pass is cheaper than a low quality cheap pass followed by repair work.

Finally, many teams optimize the wrong unit. They look at cost per API call rather than cost per successful answer. A cheaper call that causes one retry and one follow-up summarization step is not cheaper in practice. The correct operating metric is the blended cost of delivering a correct, timely, trusted answer under the service's real constraints.

Production cost therefore requires a broader lens. You are not merely buying tokens. You are buying answer quality, retrieval precision, state management, orchestration discipline, cache effectiveness, and operational predictability. The architecture decides how much of each you consume.



Illustrative figure - use the shape of the pattern, not the exact numbers, when applying this to your own system.

3. 2. The seven major sources of high cost

The first source is excessive orchestration. Teams add planners, critics, validators, reflection loops, query rewriters, and summarizers before they have proven that each stage materially improves task success. Every extra stage compounds call count, latency, and the probability that a request touches the expensive path.

The second source is prompt and context bloat. System prompts, policies, memory summaries, prior tool results, chat history, and retrieval context are all useful in moderation. In combination,

they create huge prompts that grow turn after turn. The user sees a short message; the model sees pages of repeated state.

The third source is retrieval inefficiency. Poor chunking, weak metadata, bad filters, and lack of reranking mean the generator is asked to read a large amount of irrelevant text. This burns tokens and often lowers answer quality because the real evidence gets diluted by noise.

The fourth source is model misallocation. Many systems use the strongest model for every job, even when the task is classification, extraction, query rewriting, or deterministic formatting. Premium reasoning models are then wasted on work that cheaper components could perform safely.

The fifth source is retry and repair traffic. Hallucinated answers, incomplete citations, malformed JSON, broken tool calls, and timeouts all trigger secondary calls. Some teams focus only on the main path and completely ignore the financial effect of rework.

The sixth source is repeated computation. The same document gets embedded repeatedly, the same policy block is sent every time, and the same user questions are answered from scratch. Without caching or precomputation, the system pays over and over for work it has already done.

The seventh source is poor observability. When engineers cannot see token distribution by stage, cache hit rate, retrieval budget, or escalation rate, they optimize blindly. Blind optimization usually means shaving pennies from the wrong place while the main leak continues elsewhere.

4. 3. Build a cost baseline before changing architecture

Before you optimize, you need a baseline that explains the current spend shape. Start by instrumenting every stage of the request pipeline: router, planner, retrieval, reranker, compressor, generator, validator, and fallback paths. Track input tokens, output tokens, latency, model choice, cache hit status, and whether the request eventually succeeded on the first pass.

This baseline should be segmented by request type. A search-heavy enterprise policy assistant does not behave like a chatty coding copilot or a transactional workflow bot. Group traffic into categories such as direct answer, retrieval answer, structured tool invocation, multi-step reasoning, and human escalation. Cost optimization becomes much easier once you know which category drives most of the bill.

Baseline data should also separate foreground and background work. Many teams forget to account for offline indexing, embedding refresh, batch summarization, precomputation jobs, and observability queries. Those operations may not be visible in live request logs, but they are still part of the system's total cost of ownership.

A good baseline dashboard answers a few blunt questions. How many LLM calls happen per user request? How many prompt tokens are repeated across sessions? How many retrieved chunks are actually cited in final answers? What share of traffic reaches the premium model? What percentage of output is repaired, retried, or escalated? Which documents are embedded too often?

Once these numbers are visible, optimization becomes rational. Teams can stop debating abstract best practices and start acting on evidence. In many cases, one or two levers account for the majority of avoidable spend.

Stage	Why teams add it	When it is worth it	Cost risk
Planner	Multi-step decomposition	Complex tool choice or long-horizon tasks	Unnecessary on routine questions
Critique loop	Quality rescue	High-stakes reasoning or conflicting evidence	Doubles inference if used by default
Rewrite step	Style normalization	Only if downstream contract is strict	Often replaceable by better first prompt
Validator	Reliability and safety	Cheap deterministic checks	Expensive if semantic validation is always on

5. 4. Cut orchestration before you cut model quality

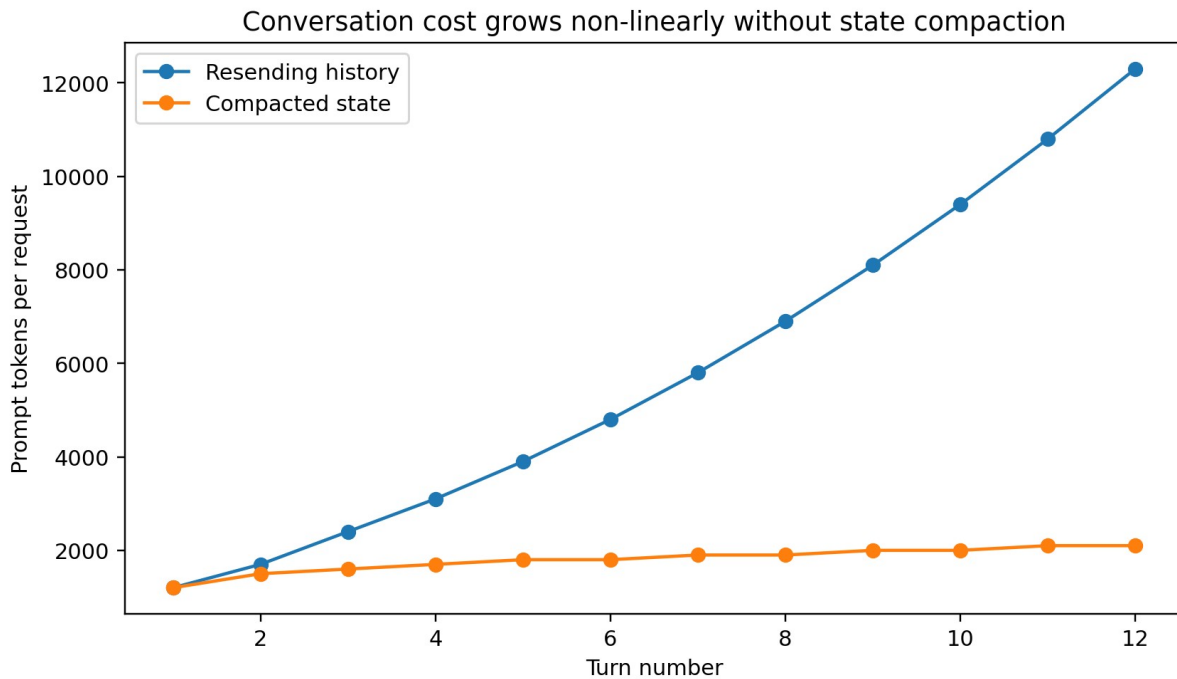
The fastest savings usually come from simplifying orchestration. Many agent stacks begin as research prototypes and evolve into production without losing their experimental scaffolding. The pipeline may include planning, decomposition, retrieval, self-critique, answer drafting, policy checking, and style rewriting, even when the majority of user requests do not need that complexity.

A disciplined production pipeline starts with a question: what is the minimum number of cognitive steps required to answer this request well? For many enterprise scenarios, the answer is three. A cheap router decides the path. Retrieval and deterministic tools gather evidence. One model synthesizes the final answer. That architecture is not simplistic; it is selective.

Self-reflection is a good example. Reflection can improve performance on difficult tasks, but it is expensive and often overused. If fewer than ten percent of queries materially benefit from a critique loop, make critique conditional instead of default. Trigger it only when confidence is low, the task is high stakes, or retrieved evidence conflicts.

Likewise, avoid letting the planner decide work that rules can decide cheaply. If a request contains a document identifier, route to a known index. If it asks for the latest policy, apply a recency filter. If it requests tabular output, enforce a schema directly. Every deterministic decision you remove from the LLM's workload saves money and improves consistency.

The goal is not to remove intelligence. The goal is to reserve expensive inference for the stages where intelligence produces outsized value. Everything else should be shrunk, routed, cached, or made deterministic.



Illustrative figure - use the shape of the pattern, not the exact numbers, when applying this to your own system.

6. 5. Context is usually more expensive than you think

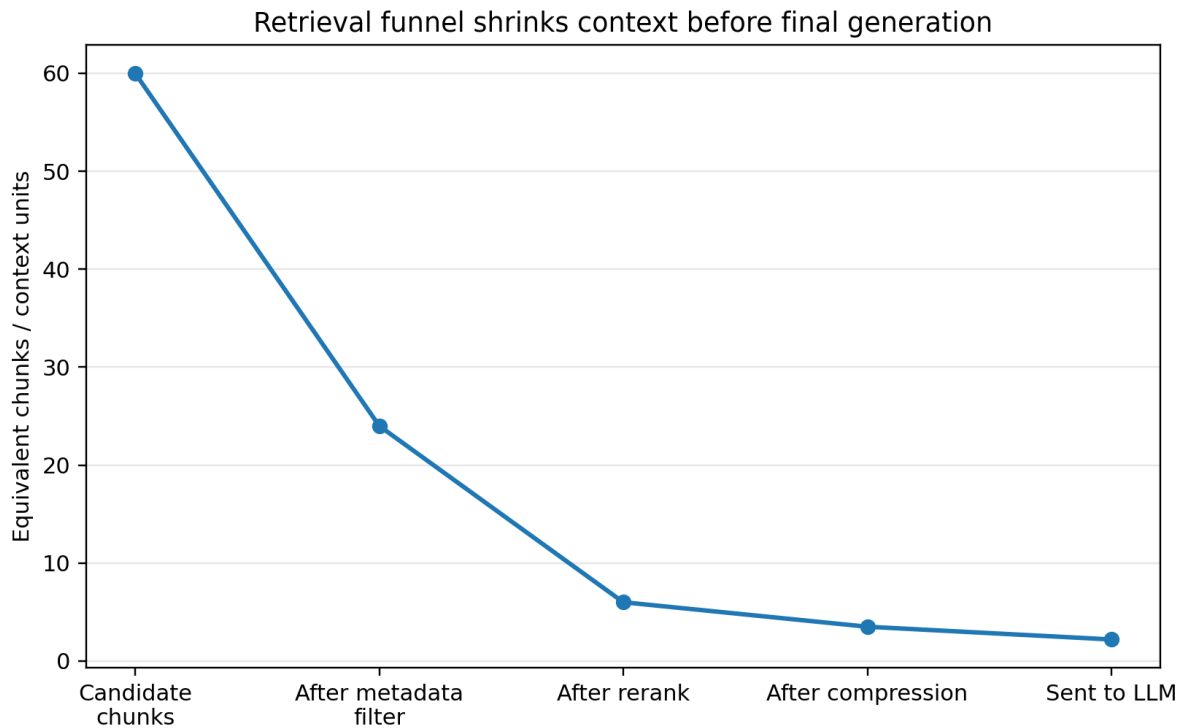
Teams often talk about model pricing as if the model is the main driver of spend. In reality, token volume is the multiplier. A powerful model with a compact, targeted prompt can be cheaper than a cheaper model fed huge amounts of repeated context. That is why prompt discipline matters as much as model choice.

The worst offender is full-history replay. Multi-turn agents frequently resend the entire conversation, previous tool traces, system instructions, and prior retrieved documents on every turn. This creates a staircase effect where each new turn becomes more expensive than the last. The user may ask a one-line follow-up, but the service sends a miniature transcript of everything that happened before.

A better approach is layered state. Keep a short live window of recent turns, a compact structured memory for persistent facts, and an external store for detailed tool outputs or artifacts. The generator should see only the information needed for the current answer. Raw traces belong in logs, not in every prompt.

Prompt templates also deserve scrutiny. Teams frequently include verbose role definitions, repeated formatting rules, and long policy blocks that barely change. If your provider supports prompt caching, separate stable prefixes from dynamic state. If not, still refactor prompts so stable instructions are concise and reusable.

Context compression is not about being stingy. It is about respecting the fact that attention is a limited budget. Smaller, more relevant prompts usually improve both cost and answer quality because the model is not forced to sift through clutter.



Illustrative figure - use the shape of the pattern, not the exact numbers, when applying this to your own system.

7. 6. RAG cost anatomy: why retrieval-heavy systems bloat

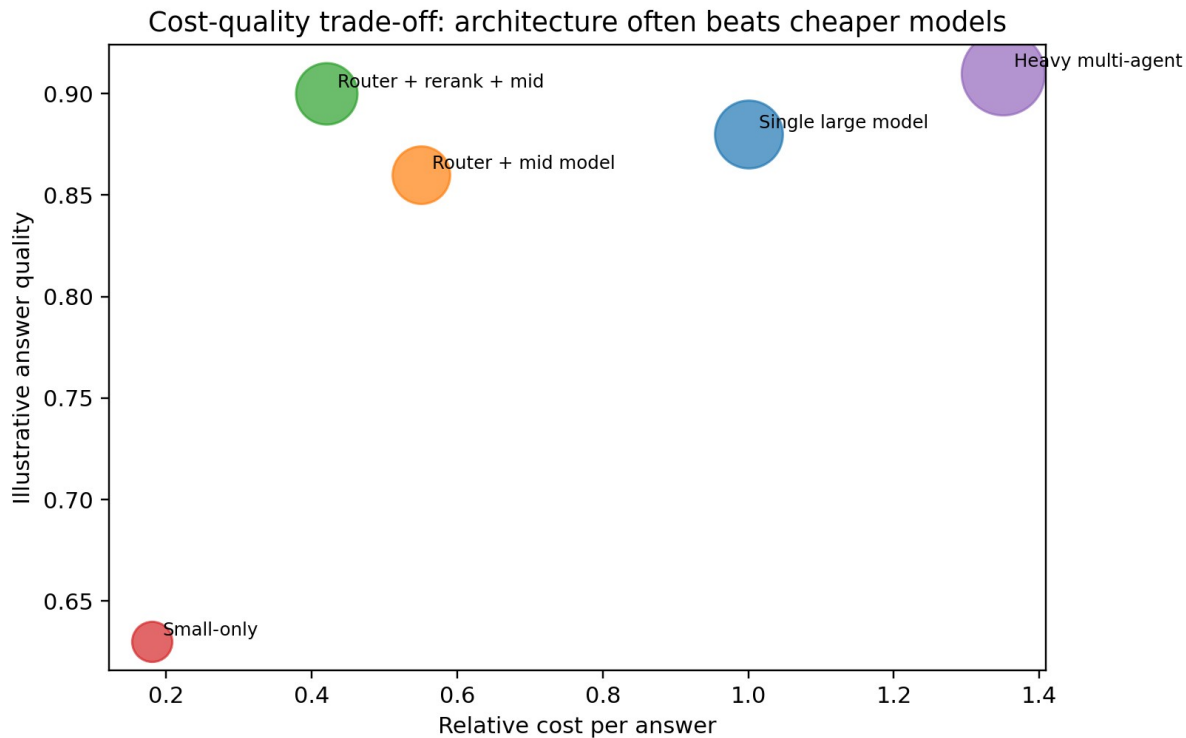
Retrieval-augmented generation often promises lower hallucination and better factual grounding, but it can become expensive when retrieval quality is poor. The generator pays for every unnecessary chunk that reaches the final prompt. If the retriever cannot separate truly relevant evidence from loosely related text, the model becomes the clean-up engine, and that is an expensive role.

The core anti-pattern is straightforward: retrieve a large number of chunks, concatenate them, and hope the model finds the right answer span. This hides weak retrieval behind a large context window. It also makes debugging harder, because no one can easily see whether the issue was retrieval, reranking, compression, or synthesis.

High-quality RAG is a funnel. Retrieval should maximize recall within a manageable search space. Metadata filtering should cut obvious noise. Reranking should identify the highest value evidence. Compression should preserve answer-bearing spans while dropping irrelevant detail. Only then should the generator synthesize. Each stage reduces the token burden placed on the final model.

Chunk design matters as much as search. Chunks that are too large waste context on irrelevant material. Chunks that are too small destroy coherence. Documents with repeated headers, tables merged into prose, or missing section metadata often behave badly in retrieval. The fix is not to retrieve more. The fix is to structure the corpus better.

When teams complain that their RAG system is expensive, the real question is usually: how much of what you retrieve is actually used? If the answer is only a small fraction, the pipeline is overpaying for noise.



Illustrative figure - use the shape of the pattern, not the exact numbers, when applying this to your own system.

8. 7. Use routing to protect premium models

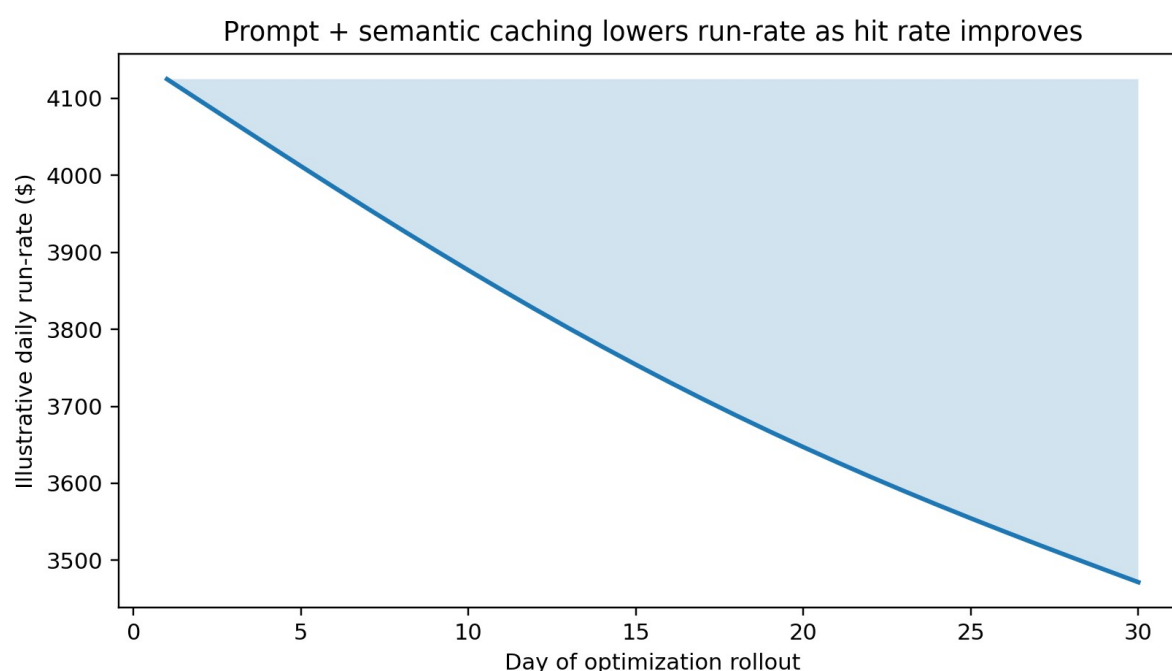
Model routing is one of the most underused cost levers because it requires confidence in task decomposition. Teams know that not every request needs their strongest model, yet they hesitate to route aggressively for fear of hurting quality. The result is a premium-first architecture that spends too much on simple work.

Start by separating tasks into capability bands. Some requests are light and deterministic: classification, sentiment tags, query normalization, metadata extraction, or parameter filling. Other requests require grounded synthesis over retrieved evidence. A smaller fraction require deep reasoning, multi-hop reconciliation, or delicate language generation. These bands do not need the same model.

A reliable routing strategy usually has at least three lanes. The cheapest lane handles lightweight text transformations and control tasks. The middle lane handles most grounded question answering. The premium lane is reserved for difficult or high-risk cases. Confidence gating then decides when to escalate from one lane to the next.

Confidence can be estimated from several signals: retriever score margin, reranker spread, output schema validity, presence of citations, contradiction across sources, uncertainty classifiers, or historical failure patterns for a query family. None of these signals is perfect alone. Together, they create a useful escalation policy.

Routing works because it changes the average price paid per successful answer. You do not need every query to take the cheapest path. You need enough routine traffic to stay off the premium path that your blended economics improve without unacceptable quality loss.



Illustrative figure - use the shape of the pattern, not the exact numbers, when applying this to your own system.

9. 8. Cache the right things, not only the final answer

When practitioners hear caching, they often think only of exact response caching: user asks the same question and the service returns the same answer. That matters, but it is only one layer. A production LLM system has multiple repeatable components, and the most effective cost reduction comes from caching at several layers.

Prompt prefix caching applies when large instruction blocks, policy text, or long document scaffolds repeat across calls. Semantic answer caching applies when different phrasings map to the same grounded answer under the same permissions and freshness policy. Retrieval caching stores

search results or candidate sets for repeated queries. Rerank caching stores the ordering of frequently reused evidence. Compression caching stores shortened passages so the same long source does not need to be trimmed over and over.

Good caching requires scope discipline. A cache key should include the parts of state that materially affect correctness: user or tenant scope, permissions, corpus version, recency requirements, and task type. Without these dimensions, caching can reduce cost while increasing the risk of stale or inappropriate responses.

The strategic advantage of caching is not only that it saves money; it also narrows latency variance. Cached paths are often much faster than live reasoning paths, which improves user experience and operational stability. This matters because lower latency reduces abandonment and often reduces the number of duplicate user requests.

Caching is therefore not a bolt-on optimization. It is part of the architecture. Teams that design for cacheability early usually outperform teams that try to retrofit caching onto unstructured prompts and loosely scoped retrieval flows.

10. 9. Precompute work outside the request path

The request path is the most expensive place to do work because it combines inference cost with user-visible latency. Any computation that can be moved offline or made incremental should be moved. This is one of the strongest production habits in cost-conscious systems.

Examples include embedding documents once instead of on demand, generating document summaries and keywords during ingestion, extracting structured metadata ahead of time, building parent-child chunk relationships, and materializing common aggregations or answer templates from trusted databases. The request path should mostly assemble already-prepared evidence, not create it from scratch.

Precomputation is especially valuable in enterprise RAG because the corpus often changes more slowly than query volume. If a policy handbook updates monthly but users ask about it thousands of times, the economics favor aggressive preprocessing. Each minute spent improving document structure and metadata upstream can save many hours of downstream inference.

The trade-off is freshness and maintenance complexity. Precomputed artifacts must be invalidated when source content changes, permissions change, or business rules evolve. That requires versioning, lineage, and a clear refresh policy. However, those operational costs are usually far lower than repeatedly asking the model to rediscover the same structure at runtime.

A simple principle helps: if the same reasoning step happens often over relatively stable data, it is a candidate for precomputation. Runtime reasoning should be reserved for truly user-specific or newly emerging questions.

11. 10. Failure loops and retries are hidden cost multipliers

Teams often undercount the money spent on things that went wrong. A malformed tool call triggers a repair call. An answer without citations triggers a validation pass. A retrieval miss triggers a second search with a rewritten query. A low-confidence answer triggers escalation. Individually these events seem minor. In aggregate they can become one of the largest cost categories in the system.

The right response is not simply to suppress retries. Some retries are necessary and good. The real objective is to reduce avoidable retries by tightening contracts between components. If the generator must emit JSON, use a schema and fail fast on invalid structure. If the answer must cite sources, require citation slots in the prompt. If the retriever often misses entities, improve metadata and lexical coverage rather than hoping that a second pass will rescue the first.

Validation itself should be targeted. Lightweight deterministic validators are ideal for schema checks, prohibited fields, citation presence, and response length controls. Save expensive semantic validation for the few cases where it truly changes decisions. Otherwise the system can end up validating everything with almost the same cost as answering everything twice.

A practical dashboard metric is retry-adjusted cost per successful answer. This metric captures the financial burden of low-quality first passes and shows whether engineering work on reliability is paying off economically. It often reveals that quality improvements can be among the strongest cost optimizations.

In other words, reliability engineering is cost engineering. A robust first pass is the cheapest path through the system.

12. 11. Observability: the control room for cost reduction

Without observability, teams end up optimizing whatever is easiest to measure, which is often not what matters most. A production-grade LLM service should emit stage-level telemetry for every meaningful step, including model choice, token counts, context size, retrieval depth, reranker usage, cache hits, latency, and result quality markers.

The telemetry model should let you answer questions at multiple resolutions. At the aggregate level, you want monthly spend trends and dominant cost drivers. At the route level, you want cost by task type and by model. At the individual request level, you want a trace showing how the request traversed the system and where tokens accumulated.

Traceability matters because cost bugs are often architectural bugs. A single misconfigured route can send a flood of routine requests to a premium model. A retrieval regression can increase average chunk count for one department's corpus. A cache invalidation bug can collapse hit rates after an index refresh. Without traces, these patterns are hard to diagnose quickly.

Observability should also link cost to outcome. Track whether the answer was accepted, whether citations were used, whether human agents reopened the issue, whether the user reformulated the same question, and whether the route escalated. Cost data without outcome data drives the wrong optimizations.

Teams that treat cost as an observability problem usually improve faster than teams that treat it as a procurement problem. Procurement can negotiate prices. Observability tells you whether the system deserves to be cheaper.

13. 12. Trade-offs: where aggressive optimization can backfire

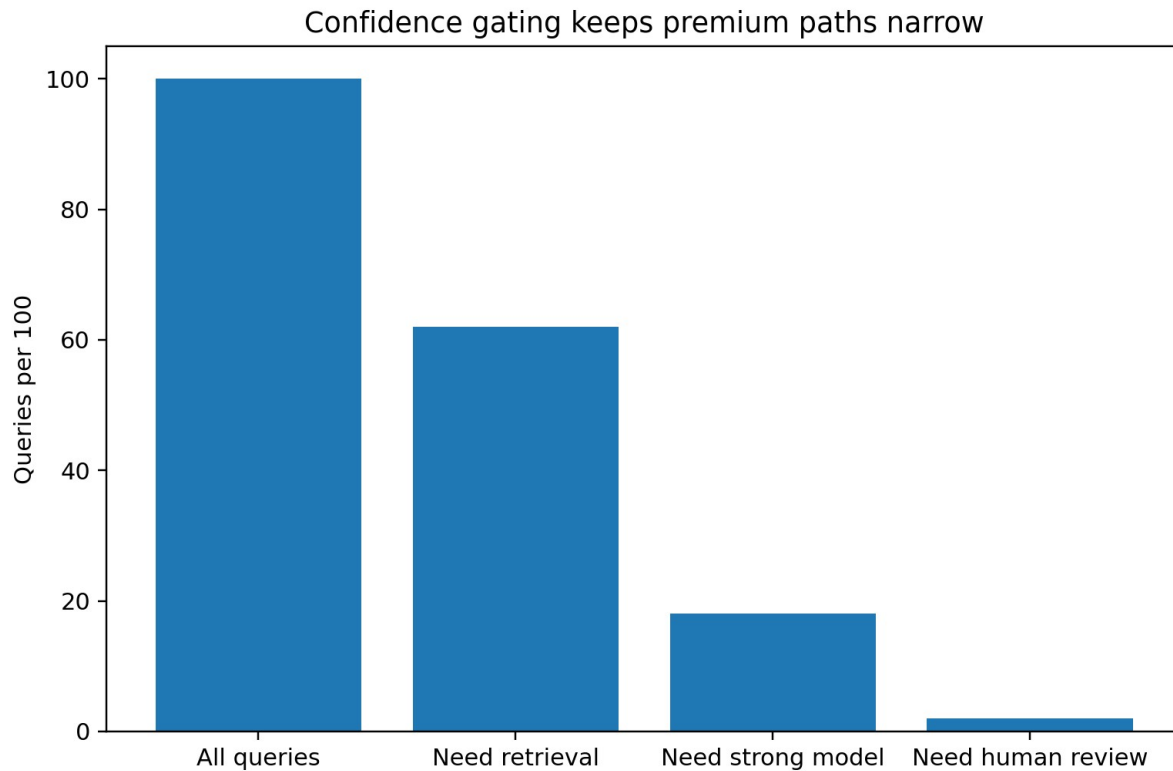
Every cost lever has a failure mode. Over-compressing context can strip away nuance. Over-routing to cheap models can reduce answer quality on borderline cases. Over-caching can serve stale answers. Over-reliance on rules can make the system brittle when inputs are ambiguous. Aggressive chunk minimization can break source coherence.

The correct posture is not maximal efficiency at any cost. It is controlled efficiency with explicit guardrails. For example, context budgets should have a soft threshold for routine questions and a higher threshold for complex comparative queries. Routing policies should include confidence triggers for escalation. Cache TTLs should be aligned with content volatility and business risk.

There is also an organizational trade-off. Sophisticated optimization can make the architecture harder to understand. A team that deploys routing, layered caches, precomputation, multiple indexes, and confidence gates must also invest in documentation, monitoring, and operational ownership. Otherwise the service becomes cheaper to run but harder to evolve safely.

A useful rule is to prefer optimizations that simplify the system while reducing cost. Reducing unnecessary agent steps, improving retrieval quality, and shrinking prompts usually satisfy this rule. Exotic optimizations should be adopted only when they solve a clearly measured problem.

Cost-conscious engineering is not about clever tricks. It is about choosing the optimizations whose complexity is justified by their savings and risk profile.



Illustrative figure - use the shape of the pattern, not the exact numbers, when applying this to your own system.

14. 13. Reference architecture for a low-cost production assistant

A practical low-cost architecture begins with a thin ingress layer that enforces authentication, rate limits, and request typing. The first cognitive component is a lightweight router that determines whether the request is direct, retrieval-based, transactional, or complex enough to warrant premium reasoning. This router also attaches basic policy and formatting requirements without emitting a full answer.

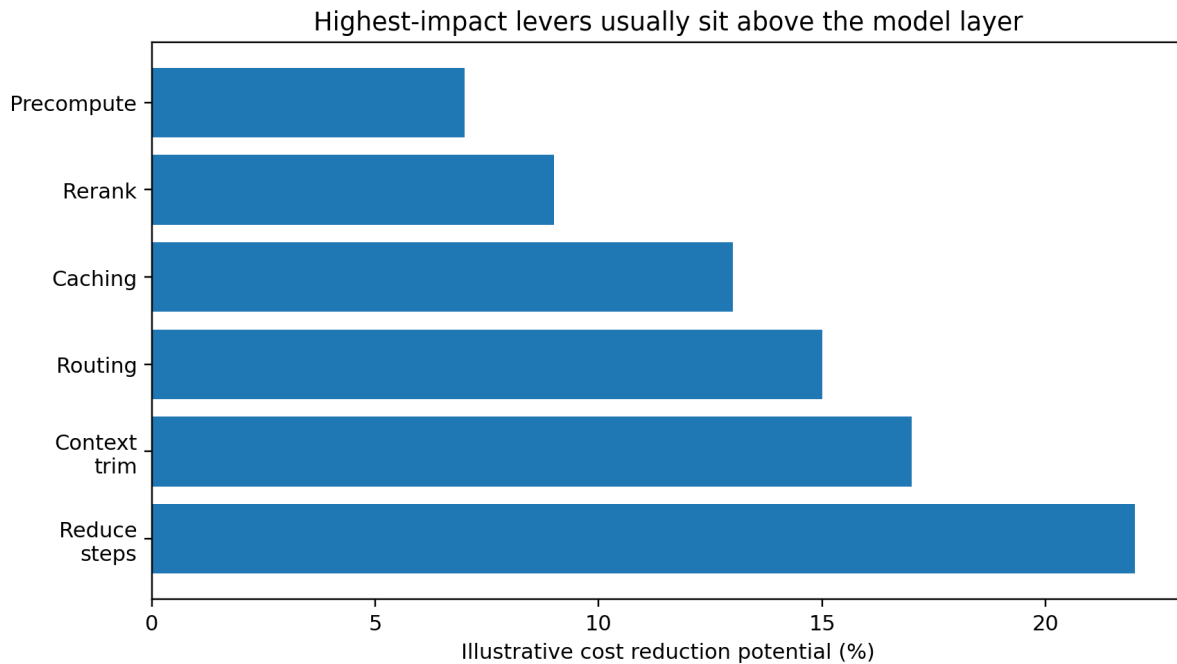
The retrieval path then applies metadata filters before any broad search. Hybrid retrieval combines lexical signals for exact entities and semantic signals for conceptual similarity. A reranker reduces the candidate set to a small number of high-value passages. A compressor extracts only answer-bearing spans and preserves source identifiers for citations.

The synthesis path uses a middle-tier model for most grounded answers. It receives only the compressed evidence, the user's active question, and a concise answer contract describing required fields or citation style. A validator checks schema, citation presence, safety conditions, and confidence. Only low-confidence or high-stakes cases are escalated to a premium model or human review.

Alongside the online path sits an offline processing plane. This plane handles chunking, embedding, document summaries, metadata extraction, answer template materialization, and refresh jobs. It is where most expensive but reusable document preparation work should live. The online service then becomes thinner, cheaper, and easier to reason about.

The architecture is complemented by a multi-layer cache and a trace store. Together, these components ensure that repeated work is reused and that every expensive path can be audited later. The system is not cheap because any single model is cheap. It is cheap because expensive work happens rarely and only when it must.

Maturity	Primary goal	Best levers	Watch-outs	Expected outcome
Early	Stop obvious waste	Step reduction, context caps, token visibility	Do not optimize blindly	Fast first cost drop
Intermediate	Improve blended efficiency	Routing, rerank, compression, caching	Need route-level testing	Lower cost without large quality hit
Mature	Stabilize economics	Offline preprocessing, confidence gates, drift monitoring	Operational complexity	Predictable cost per successful answer



Illustrative figure - use the shape of the pattern, not the exact numbers, when applying this to your own system.

15. 14. Engineering checklist by maturity stage

At the earliest stage, focus on visibility and obvious waste. Count calls per request. Measure prompt size by stage. Track top documents sent into final prompts. Limit retrieval depth. Add a hard cap on context budget. Remove gratuitous critique loops. These changes are usually enough to produce the first meaningful cost drop.

At the intermediate stage, introduce routing, reranking, context compression, and cache segmentation. Improve chunking quality, attach metadata consistently, and separate live state from durable memory. Begin tracking retry-adjusted cost per successful answer. Create dashboards that compare request categories rather than looking only at global totals.

At the mature stage, move repeated reasoning into offline jobs, add confidence-based escalation, support parent-child retrieval, and tune budget policies by query type. Measure not just cost and quality, but also volatility: how often does one release change average token usage or hit rates? Mature systems manage economics as a living property of the architecture, not as a quarterly clean-up exercise.

Across all stages, insist on three principles. First, every extra step must justify itself. Second, every token must earn its place in context. Third, every expensive path must be observable. These principles are simple, but they separate research-style pipelines from durable production systems.

The checklist should also include governance. Define who owns prompt changes, retrieval schema, cache invalidation, cost dashboards, and fallback logic. Cost rises fastest in systems where everyone can add complexity and no one owns the economics.

16. 15. Code patterns that reduce cost in practice

The remaining pages of this guide include example code patterns for the most common optimization levers. The exact framework is less important than the design intent. You can implement these patterns in almost any stack as long as you preserve the same invariants: explicit routing, bounded context, reusable retrieval artifacts, and narrow escalation.

The examples cover a request budget ledger, a lightweight router, a retrieval funnel with metadata filtering and reranking, prompt compaction, semantic caching, confidence-based escalation, and a telemetry envelope for measuring cost per successful answer. None of these snippets is meant to be copy-pasted unchanged into production, but each captures a design shape that consistently reduces spend.

A common error is to adopt isolated code snippets without changing the service contract around them. For instance, a semantic cache will fail if permissions and corpus versions are not part of the key. A reranker will disappoint if the candidate pool is poorly chunked. A router will misbehave if the task taxonomy is vague. Code helps, but architecture gives the code meaning.

As you review the examples, notice the theme: most savings come from discipline at the boundaries between steps. The service decides early, passes compact state, validates cheaply, and escalates reluctantly. That pattern is more valuable than any single library call.

The code is therefore an implementation appendix to the core message of the guide: lower cost is mostly the product of better design decisions made earlier in the pipeline.

17. 16. Detailed playbook: reason, symptom, fix, trade-off

Excessive step count typically reveals itself through traces that show three to seven model calls for routine questions. The symptom is a high average cost even when the questions themselves are simple. The root cause is often that experimental planning and reflection stages were never retired. The fix is to map each stage to measurable value, remove anything that does not materially improve accuracy, and make advanced reasoning loops conditional rather than default. The trade-off is that some corner cases may initially lose a small amount of quality until confidence-based escalation is tuned properly.

History replay becomes visible when prompt tokens per turn grow faster than user message length. The symptom is that late-session follow-ups become disproportionately expensive. The root cause is that the service replays the whole conversation, old tool traces, or verbose summaries. The fix is to move detail into external state, keep a short active window, and maintain a compact structured memory for persistent facts. The trade-off is implementation effort: developers must define what belongs in memory and what should stay in logs.

Retrieval waste shows up when many chunks are retrieved but only one or two are cited or meaningfully referenced. The symptom is high prompt size with no corresponding lift in answer quality. The root cause is weak chunking, poor filters, or a missing reranker. The fix is to improve corpus structure, filter first, rerank aggressively, and compress evidence before generation. The trade-off is extra engineering around indexing and relevance evaluation.

Premium model overuse is easiest to detect by looking at route share. If nearly all traffic goes to the most expensive model, routing is not working. The fix is to classify requests by task complexity and define escalation signals. The trade-off is that route design must be tested carefully; otherwise a cheap lane can become a quality trap for requests that sit near the boundary.

Repeated computation is visible when the same documents, policies, or answers are processed repeatedly across users and sessions. The fix is caching and precomputation. The trade-off is that scope, freshness, and permission checks must be built correctly, otherwise savings come with correctness risk.

Retries and repair loops show up when one user request leads to multiple hidden internal calls. The fix is stronger contracts between components, more deterministic validation, and better first-pass grounding. The trade-off is that stricter validation may surface failures more explicitly in the short term, which can feel like a regression even though it is actually a visibility improvement.

Weak observability shows up when teams argue about likely causes instead of presenting route-level evidence. The fix is telemetry design: traces, metrics, and cost accounting per successful answer. The trade-off is that instrumentation takes time, but the return on that time is unusually high because it guides every later optimization.

18. 17. Cost metrics every production team should track

The first metric is average LLM calls per resolved request. This number immediately exposes orchestration creep. It should be segmented by route because a complex planning workflow may be appropriate for one route and absurd for another.

The second metric is prompt tokens and completion tokens by stage. Looking only at total tokens hides whether cost is happening in retrieval synthesis, critique loops, or routing. Stage-level token accounting often produces the clearest optimization roadmap.

The third metric is retrieval efficiency. At minimum, track retrieved chunk count, post-rerank chunk count, compressed context size, and citation utilization. Citation utilization can be approximated by measuring what share of final cited evidence came from the retrieved set. Low utilization usually signals a noisy retrieval pipeline.

The fourth metric is premium model share. This should include both direct premium routing and premium escalation from cheaper paths. A rising premium share is not always bad, but it should have a visible explanation such as a new high-stakes workflow or deteriorating retrieval quality.

The fifth metric is cache hit rate at each layer: prompt prefix, retrieval, rerank, compression, and answer cache. Aggregate cache hit rate is less useful than layer-specific visibility because it tells you where reuse is and is not happening.

The sixth metric is retry-adjusted cost per successful answer. This is the most economically honest KPI because it incorporates failures, repairs, and escalations. If this metric falls while acceptance remains stable, your optimization program is working.

The seventh metric is stale-answer risk. Track cache invalidation lag, corpus refresh lag, and answer age for freshness-sensitive routes. Cheap answers are not useful if they are wrong because they were generated from old evidence.

The eighth metric is operational volatility. Measure how much average token usage, cache hit rate, or route share changes after each release. Volatility is a cost risk because it means economics can drift quickly without detection.

19. 18. Common anti-patterns and what to do instead

Anti-pattern one: using the generator to compensate for bad retrieval. Teams retrieve large contexts and hope the LLM can infer the relevant part. Replace this with metadata filters, reranking, and passage compression.

Anti-pattern two: forcing every request through a generic agent loop. If the product has stable workflows, encode them as routes or tool policies rather than rediscovering them with the model each time.

Anti-pattern three: storing memory as prose. Narrative summaries are harder to control, longer to replay, and less reusable. Replace them with compact structured state plus targeted retrieval of detailed history when needed.

Anti-pattern four: treating tool traces as part of the conversational prompt. Tool traces belong in logs or external state. The final model usually needs conclusions or selected outputs, not the entire execution trace.

Anti-pattern five: caching only exact final answers. Many opportunities sit earlier in the pipeline. Cache candidate sets, reranked sets, compressed passages, and document summaries.

Anti-pattern six: measuring cost only monthly. Monthly reports are too slow for a fast-moving service. Use daily route-level dashboards and trace sampling so regressions are visible quickly.

Anti-pattern seven: optimizing context after quality declines. Context budgets should be designed intentionally from the beginning, otherwise teams end up paying for large prompts while quality still remains unstable.

Anti-pattern eight: assuming that a larger context window removes the need for retrieval discipline. Large windows are a convenience feature, not a substitute for precise evidence selection.

20. 19. Example implementation patterns

The code below illustrates a budget ledger that follows one request through the pipeline. In production, this object would likely be attached to a distributed trace and enriched by middleware. Its purpose is simple: make every hidden stage visible and computable.

The router example shows a pragmatic pattern. Use a cheap control model or even deterministic rules for obvious intents, and reserve semantic classification for genuinely ambiguous questions. That single design change often removes a surprising amount of waste.

The retrieval example demonstrates a funnel rather than a giant concatenation step. Metadata filters narrow scope, hybrid search improves candidate quality, reranking reduces context, and compression extracts the smallest answer-bearing spans that preserve grounding.

The semantic cache example stresses key design. Similarity alone is not enough. The key must also include tenant scope, permission scope, corpus version, freshness profile, and route identity. A cache that ignores these dimensions may save money while increasing risk.

The escalation example keeps the premium path narrow by combining retrieval confidence, schema validity, and source agreement into a simple decision function. It is intentionally conservative: better to escalate a modest minority of uncertain cases than to let all traffic touch the premium lane.

21. 20. Final recommendations

If you are struggling to reduce cost in production, resist the urge to begin with provider comparisons. Provider price differences matter, but architectural waste usually dominates first. Start by understanding where calls and tokens are being spent. Then reduce orchestration, shrink context, improve retrieval precision, route work by difficulty, cache repeated computation, and move reusable reasoning offline.

Do not frame this as a war between quality and efficiency. In well-designed systems, better grounding, better routing, and better state management improve both. The expensive architecture is often not the most accurate architecture; it is merely the least disciplined one.

When deciding what to tackle next, choose levers in this order. First remove unnecessary steps. Then reduce context volume. Then protect premium models with routing. Then add reranking and compression. Then layer caching and precomputation. Finally, invest in observability and maturity practices so savings remain durable.

A production team that follows this order will usually discover that cost reduction is not a one-time exercise. It becomes part of the engineering culture. New features are evaluated not only for usefulness, but also for whether they create repeated work, oversized prompts, or hidden premium dependencies.

That is the real outcome of this playbook. Not merely a cheaper bill, but a service whose economics are understandable, controllable, and aligned with the value it delivers.

22. 21. Playbook for enterprise knowledge assistants

Enterprise knowledge assistants usually serve policies, procedures, support articles, compliance documents, and internal manuals. Their cost profile is dominated by retrieval and repeated questions rather than raw open-ended creativity. That makes them excellent candidates for strong metadata design, answer caching, and upstream corpus preparation.

Begin with a document ingestion contract. Every document should carry owner, department, date range, confidentiality level, title, section hierarchy, and version metadata. Many teams skip this because the content appears searchable without it. In production, missing metadata forces the retriever to search too broadly and increases the burden on reranking and generation.

Chunk boundaries should follow semantic structure. Sections, subsections, FAQs, procedures, and tables should be represented deliberately rather than split with a purely mechanical window. Enterprise users often ask for rules, exceptions, or the latest version of a policy. Those questions benefit from chunks that preserve scope and version context.

For enterprise assistants, answer contracts matter. Require the generator to provide a concise answer, source references, and a statement of applicability when useful, such as policy version or geography. This not only improves trust but also lowers the chance of expensive follow-up questions asking where the answer came from or whether it applies in a specific region.

A particularly strong optimization in this setting is canonical answer precomputation for the top recurring intents. If employees ask the same hundred questions every week, you should not treat every instance as a fresh reasoning challenge. Retrieve and refresh those answers in a controlled background process, and use live retrieval only when freshness or nuance requires it.

The trade-off is maintenance discipline. Owners must be assigned to source versioning and refresh schedules. But where knowledge content is relatively stable, the savings and reliability gains are significant.

Another useful tactic is audience-aware context reduction. A service desk agent may need procedural detail, while an executive may only need a concise policy summary and citation. Tailoring answer depth to user role can reduce completion tokens materially without harming usefulness.

Finally, enterprise assistants benefit from strong permission-aware caching. Two users asking the same question may not have the same access rights. Cache keys therefore need authorization scope, not just semantic similarity.

23. 22. Playbook for coding assistants and developer agents

Developer-facing assistants have a different cost profile. They often combine long conversational context, repository retrieval, tool traces, and iterative refinement. The risk here is not only token bloat but also expensive back-and-forth loops where the assistant proposes a change, the user rejects it, and the model tries again with a new patch.

The first optimization is to separate planning from patch generation. Do not send the whole repository or execution trace into the main prompt by default. Search files first, summarize relevant symbols and diffs, and present the generator with the smallest workable context.

Second, use syntax-aware chunking and retrieval. Code chunks should preserve function boundaries, imports, signatures, and neighboring lines likely to affect correctness. Naive fixed-size chunks create irrelevant context and push the model to reason over fragments rather than coherent code units.

Third, compress tool outputs aggressively. Build logs, test results, and stack traces are often long, repetitive, and only partially informative. Extract failing test names, error types, changed files, and the narrowest surrounding evidence. Tool output summarization alone can save large numbers of tokens in iterative debugging workflows.

Fourth, treat patch validation as deterministic whenever possible. Unit tests, static analysis, formatting checks, and schema constraints are cheaper and more reliable than asking another LLM to inspect code for correctness. Use the model to interpret failures selectively, not to replace the tooling ecosystem.

Fifth, cache repository summaries, file embeddings, and symbol graphs. Developer agents frequently revisit the same modules during a session. A good local state model can prevent repeated retrieval and repeated explanation of unchanged code structures.

The trade-off in coding workflows is that aggressive compression can remove details needed for a correct fix. The safest compromise is to retrieve broad evidence cheaply, then pass only the minimal code neighborhoods into generation, while leaving the full repository accessible for deterministic verification.

Developer agents also benefit from explicit budget policies. For example, limit high-cost reasoning paths to bug triage, architectural changes, or uncertain merges, while keeping routine code explanations and test result summaries on a cheaper lane.

24. 23. Playbook for workflow and transactional agents

Workflow agents interact with business systems: creating tickets, updating records, generating reports, checking status, or orchestrating task sequences. Their cost problem is often self-inflicted because teams let an LLM decide procedural steps that could have been expressed as deterministic workflows with parameter extraction.

In these agents, the highest-value LLM role is usually interpretation at the boundary: understanding the user intent, extracting parameters, and handling ambiguity. Once intent and parameters are known, the sequence of actions should often be deterministic, auditable, and tool-driven.

For example, if a user asks to create a vendor onboarding ticket, the agent should not repeatedly reason about which system to touch, what fields are mandatory, or whether a due date is needed. Those rules belong in workflow definitions and schemas. The model should fill the fields and ask targeted questions only when required data is missing.

This pattern lowers cost because it replaces open-ended chain-of-thought style orchestration with structured execution. It also reduces risk, which is important in systems that mutate data or trigger downstream operations.

A good optimization in transactional agents is parameter memory. If the user already confirmed a project code or region earlier in the session, store it as structured state instead of making the model re-derive it from chat history. This reduces both prompt size and accidental inconsistencies.

Another strong lever is idempotent retry design. When tool failures occur, retries should not require the entire conversational context to be replayed. Preserve the execution state, the failed step, and the relevant parameters. Then either retry deterministically or ask the model for a narrow repair action.

The main trade-off is flexibility. Deterministic workflows handle common tasks beautifully but may feel rigid for unusual or cross-process requests. The solution is not to abandon structure. It is

to include a deliberate escape hatch that hands rare ambiguous cases to a more capable reasoning path.

A workflow agent becomes economical when the LLM interprets, but the system executes.

25. 24. Chunking and corpus design in depth

Chunking is often discussed as a tuning parameter, but in production it is closer to a document design problem. The structure you impose on the corpus determines what the retriever can find cheaply and what the generator must reassemble at runtime.

The simplest guideline is to align chunks to semantic boundaries whenever possible. A policy definition, a numbered procedure, an FAQ answer, a product limitation, and a table note are different kinds of units. They should not all be treated as generic text windows.

Overlap should be used intentionally rather than reflexively. Small overlaps can preserve continuity across boundaries, but heavy overlap increases storage volume, retrieval duplication, and the chance that near-identical chunks flood the top results. If the retriever often returns overlapping siblings, the generator ends up paying to read the same information twice.

Tables deserve special treatment. Many pipelines flatten them poorly, merging row and column semantics into unreadable text. When users ask for eligibility rules, thresholds, or mapping tables, poorly extracted tables become a major cause of retrieval misses and generator confusion. Consider representing tables both as structured data and as human-readable textual explanations.

Parent-child chunking is especially powerful in long manuals and complex policy documents. Small child chunks optimize retrieval precision, while parent chunks preserve context for synthesis when necessary. This lets the service retrieve narrowly while still expanding gracefully for nuanced questions.

Add metadata not only for filtering but also for explanation. Section title, document type, effective date, and region are not merely search aids; they help the final answer provide useful citations and applicability notes without extra reasoning.

The trade-off in sophisticated chunking is pipeline complexity. Ingestion takes more care, and corpus refreshes must maintain relationships consistently. Yet for large, high-query corpora, chunk quality often determines whether the service can stay both cheap and accurate.

If your RAG system keeps needing more context to perform acceptably, the problem may begin upstream in corpus design rather than downstream in generation.

26. 25. Retrieval evaluation in depth

Teams often optimize retrieval informally by looking at a few example questions. That is not enough. Retrieval needs its own evaluation set, because a generator can sometimes compensate for weak evidence and create the illusion that search quality is adequate.

Build a test set with representative queries, expected source documents, and ideally expected answer-bearing spans. Include exact-entity queries, paraphrased conceptual queries, date-sensitive questions, and multi-document comparison questions. These query families stress different parts of the retrieval stack.

Measure recall at candidate depth, reranked precision, citation coverage, and final answer grounding. A retriever that finds the right document at rank thirty may be acceptable if the reranker consistently promotes it. A retriever that finds semantically related but non-authoritative documents may look plausible in demos yet cause expensive answer repair later.

Metadata filters deserve evaluation too. Filters can dramatically reduce cost by narrowing search space, but overly strict filters can hide the correct evidence. Evaluate both filtered and unfiltered paths on representative traffic before locking in defaults.

Human review remains valuable for retrieval because relevance is contextual. A chunk can be topically related but still unusable for a precise answer. Periodic expert review of failure cases helps refine chunking, metadata, lexical features, and rerank prompts or models.

Track retrieval drift over time. As documents change, new formats are added, or indexing rules evolve, retrieval quality can regress even if model behavior stays constant. Drift monitoring protects both quality and cost because regressions often surface first as context inflation or rising premium escalation.

The trade-off is evaluation maintenance. However, without retrieval evaluation, teams are forced to debug cost through trial and error, which is slower and more expensive.

A well-measured retrieval layer pays for itself by allowing the final generator to stay lean.

27. 26. Memory design in depth

Memory in agentic systems is frequently implemented as a generic conversation summary. While better than raw history replay, that pattern still leaves money on the table because it mixes stable facts, temporary state, tool outputs, and conversational nuance into one undifferentiated blob.

A more economical memory model separates three layers. The first is durable profile memory: long-lived facts about the user, project, or workflow. The second is task state: the current objective, constraints, selected tools, and unresolved questions. The third is evidence state: references to documents, tool outputs, or intermediate results that can be pulled in on demand.

Each layer has a different volatility and therefore a different storage and replay strategy. Durable profile memory changes slowly and should be concise. Task state changes during a session and should be updated structurally. Evidence state can be large and should mostly live outside the prompt, referenced by identifiers until needed.

This layered approach reduces prompt growth because the generator receives only the slice relevant to the current step. It also improves reliability because the system knows which facts were user-confirmed, which were inferred, and which came from external tools.

Memory should also be budgeted. For example, profile memory might be capped at a few hundred tokens, task state at a few hundred more, and evidence state referenced externally unless the next step explicitly requires it. Without explicit budgets, memory grows by convenience rather than design.

The trade-off is implementation complexity. Developers need to define state schemas and update rules. Yet once in place, layered memory usually lowers both cost and behavioral drift in long sessions.

A useful principle is that memory should improve decision quality, not merely preserve narrative continuity. When memory stops helping decisions, it has become prompt baggage.

The best memory systems are not the longest; they are the most discriminating.

28. 27. Governance and FinOps for LLM systems

Cost optimization is not only a technical exercise. It is also a governance problem. Without clear ownership, teams add prompts, routes, and models freely while no one is accountable for the economics of the service.

A practical governance model defines owners for prompts, retrieval schema, routing policy, caches, telemetry, and provider usage. Changes to any of these areas should be reviewable and measurable. Prompt edits, for example, can meaningfully change token usage, route share, and validation failure rates.

Introduce lightweight release gates for cost-sensitive systems. A route change should show expected token budgets. A new corpus should include metadata and chunking rules. A new model should be benchmarked on both quality and cost. These gates do not need to be bureaucratic; they need to prevent silent regressions.

FinOps practices help translate engineering metrics into business language. Instead of saying that prompt tokens rose by twenty percent, present the equivalent increase in blended cost per thousand resolved requests, or the projected monthly impact of a route misconfiguration. This makes cost conversations easier with leadership and platform teams.

Budget alerts should be tied to service behavior, not just invoice totals. Rising premium share, falling cache hit rate, or sudden growth in average retrieved chunks are earlier warning signs than the monthly bill. Early warnings enable corrective action before costs harden into recurring spend.

Governance must also include data freshness and permission risk. A cost-saving mechanism that serves stale or unauthorized data creates a different kind of liability. FinOps for LLM systems therefore sits alongside reliability, compliance, and knowledge management, not beneath them.

The trade-off is process overhead, but the right level of process is usually far cheaper than uncontrolled architectural drift.

A system with no cost ownership is usually expensive by default.

29. 28. Migration plan: how to optimize without breaking production

Teams rarely get the chance to rebuild from scratch. Most must optimize a live system that users already depend on. The safest approach is incremental migration with route-level controls, shadow evaluation, and reversible feature flags.

Phase one is visibility. Add stage-level telemetry, estimate current route costs, and identify the top two or three waste patterns. Do not change everything at once. First make the invisible visible.

Phase two is no-regret simplification. Remove duplicate prompts, cap context budgets, narrow retrieval depth, and disable clearly unnecessary critique loops. These changes generally reduce cost with minimal product risk.

Phase three is controlled architectural improvement. Introduce routing for a small subset of traffic, add reranking behind a flag, and deploy context compression on read-only knowledge routes before enabling it on more complex workflows. Compare quality and cost in parallel against the current production path.

Phase four is cache layering. Start with prompt prefix or retrieval caches where correctness risk is low and observability is strong. Then expand to semantic answer caching only after key design and freshness controls are well tested.

Phase five is offline movement. Shift repeated summarization, metadata extraction, and corpus preparation out of the live path. Because these changes affect data pipelines and refresh jobs, they should follow clear ownership and versioning rules.

Throughout migration, define rollback triggers. For example, if answer acceptance falls beyond a threshold or stale-answer incidents rise, disable the specific optimization rather than reverting the whole platform. This keeps the program controlled and credible.

The migration trade-off is speed versus safety. Fast cost reduction is tempting, but trust lost through production regressions is expensive to regain. A disciplined migration program protects both economics and confidence.

A good migration plan makes optimization feel boring in the best possible way: measurable, gradual, and unsurprising.

30. 29. Case-study style scenarios

Scenario one: an internal policy assistant shows high cost despite modest traffic. Investigation finds that every question uses the premium model, retrieves fifteen chunks, and sends a long policy instruction block on every call. The fix sequence is simple: add routing, separate stable prompt prefixes, introduce reranking, and compress evidence. Result: lower blended cost with better citation quality because the final model sees less clutter.

Scenario two: a developer agent becomes expensive during debugging sessions. Traces show large build logs and long code histories being replayed at each iteration. The fix is to summarize tool outputs deterministically, retrieve only changed file neighborhoods, and store execution state externally rather than in chat history. Result: lower latency and fewer unnecessary tokens per turn.

Scenario three: a workflow bot for ticket creation has low direct token cost but high retry cost because the model often misses required fields, causing repair loops. The fix is to move field requirements into structured schemas and only ask targeted follow-up questions for missing values. Result: the first pass becomes cheaper by becoming more deterministic.

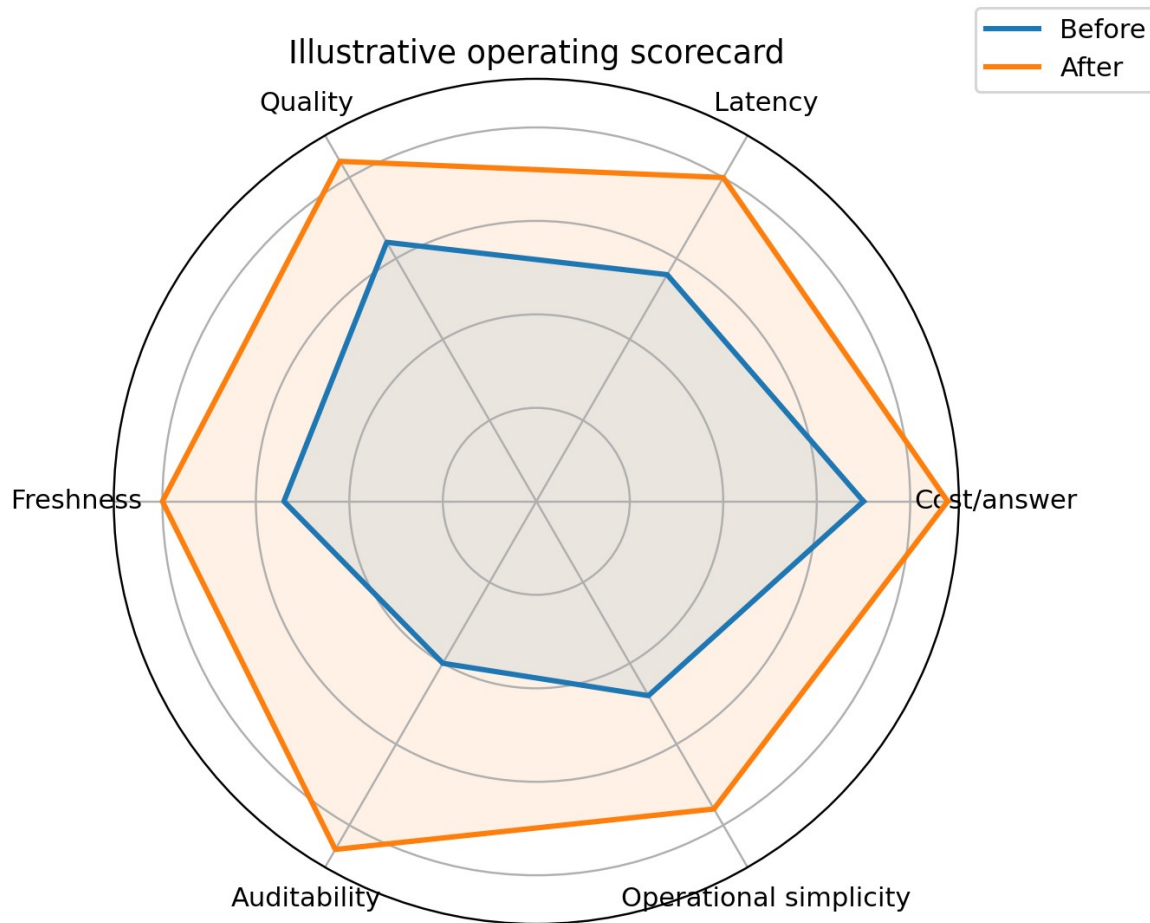
Scenario four: a multi-tenant knowledge service introduces semantic caching but serves a few stale responses after policy updates. The fix is not to abandon caching; it is to include corpus version and freshness profile in cache keys and to invalidate entries when the source lineage changes. Result: cost savings are preserved with lower correctness risk.

Scenario five: an executive dashboard assistant appears cheap per call but expensive per successful answer because business users frequently ask follow-up questions. Analysis reveals that answers are technically correct but too verbose and lack direct recommendations. The optimization is not a cheaper model. It is a better answer contract that produces concise, decision-ready outputs and reduces repeat traffic.

These scenarios matter because they show a consistent pattern. High cost is rarely caused by one dramatic mistake. It is usually the sum of several ordinary design decisions that were never revisited under production load.

The operational lesson is to diagnose by trace and route, not by intuition. Once you can see the shape of each scenario, the cost levers become easier to prioritize.

A team that can recognize these patterns quickly will save both money and engineering time.



Illustrative figure - use the shape of the pattern, not the exact numbers, when applying this to your own system.

31. 30. What good looks like

A well-optimized production LLM service has a recognizable feel. Request traces are short and intelligible. Most routine traffic takes a compact path. Retrieval context is small enough to inspect manually. Premium models are used selectively and defensibly. Cache hit rates are stable. Releases show predictable economic effects rather than surprises.

Users experience this service as responsive and consistent rather than merely cheap. Answers arrive quickly, cite the right sources, and avoid unnecessary verbosity. Follow-up questions are

fewer because the first answer is shaped for the user's real need. Human support teams trust the outputs because they can see how the system arrived there.

Engineering teams experience it as operable. When cost rises, they can identify which route changed. When quality drops, they can determine whether the cause sits in retrieval, routing, prompts, or data freshness. Optimization becomes a regular engineering practice instead of a reactive emergency.

Leadership experiences it as a platform whose economics make sense. Spend grows with value delivered rather than with hidden complexity. Capacity planning becomes easier because cost per resolved request is stable enough to model. Vendor negotiations become more productive because the team knows its own architecture well.

What good looks like is therefore broader than a lower bill. It is an architecture where money, quality, and operational behavior all move in understandable ways.

That is the ultimate aim of this guide. Not a bag of tricks, but a production discipline for running agentic and retrieval-augmented systems sustainably.

Once a team reaches this state, further optimization becomes easier because the system is no longer fighting itself.

And that, more than any single model choice, is what keeps LLM systems affordable at scale.

32. Appendix A. Annotated code walkthroughs

The following code snippets are deliberately compact, but each one illustrates a production principle. Read them less as full applications and more as architectural patterns that can be adapted to your stack.

The budget ledger pattern ensures that a request cannot move through the system invisibly. Every stage records tokens, model, latency, and notes. Once this object is attached to traces and persisted for analysis, cost conversations stop relying on guesswork.

The router pattern shows that you do not need expensive reasoning to decide whether a request is a greeting, a structured workflow command, a retrieval query, or a complex analysis task. In many organizations, deterministic heuristics plus a small control model are enough for the majority of routing decisions.

The retrieval funnel pattern illustrates how each stage narrows scope. Filters remove obviously irrelevant data. Hybrid retrieval improves candidate quality. Reranking concentrates signal. Compression removes token bulk before synthesis. When all four work together, the final prompt becomes dramatically smaller.

The semantic cache pattern demonstrates why key design is the heart of safe reuse. Scope, freshness, route identity, and permissions must travel with semantic similarity. A cache that only knows the words of the question is not a production cache; it is a liability.

The confidence gate pattern reminds us that escalation should be evidence-based. Confidence is not a magical single number. It is an aggregate view drawn from retrieval quality, source agreement, schema validity, and task sensitivity. This kind of gate protects premium paths while preserving quality on uncertain cases.

Budget ledger

```
from dataclasses import dataclass, field
from time import perf_counter

@dataclass
class StageRecord:
    name: str
    model: str
    prompt_tokens: int = 0
    completion_tokens: int = 0
    latency_ms: int = 0
    notes: dict = field(default_factory=dict)

@dataclass
class RequestLedger:
    request_id: str
    route: str
    stages: list[StageRecord] = field(default_factory=list)

    def add_stage(self, name, model, prompt_tokens, completion_tokens, latency_ms, **notes):
        self.stages.append(
            StageRecord(
                name=name,
                model=model,
                prompt_tokens=prompt_tokens,
                completion_tokens=completion_tokens,
                latency_ms=latency_ms,
                notes=notes,
            )
        )

    @property
    def total_tokens(self) -> int:
        return sum(s.prompt_tokens + s.completion_tokens for s in self.stages)

    @property
    def total_latency_ms(self) -> int:
        return sum(s.latency_ms for s in self.stages)
```

Why it matters: This pattern reduces hidden waste by making decisions explicit, bounding context, or enabling safe reuse. In production, adapt the schema, persistence layer, and error handling to your environment before deploying.

Rule-first router

```
def route_request(user_text: str) -> str:
```

```

text = user_text.lower().strip()

if text in {"hi", "hello", "thanks"}:
    return "small_talk"

if "create ticket" in text or "raise request" in text:
    return "workflow"

if any(word in text for word in ["policy", "document", "manual", "latest rule"]):
    return "rag_search"

if len(text.split()) < 8:
    return "cheap_direct"

return "reasoned_answer"

```

Why it matters: This pattern reduces hidden waste by making decisions explicit, bounding context, or enabling safe reuse. In production, adapt the schema, persistence layer, and error handling to your environment before deploying.

Retrieval funnel

```

def retrieve_context(query, tenant_id, filters):
    candidates = hybrid_search(
        query=query,
        tenant_id=tenant_id,
        metadata_filters=filters,
        top_k=40,
    )

    reranked = rerank(query=query, passages=candidates, top_k=6)

    compressed = [
        compress_passage(query=query, passage=p["text"], max_tokens=180)
        for p in reranked
    ]

    return compressed[:4]

```

Why it matters: This pattern reduces hidden waste by making decisions explicit, bounding context, or enabling safe reuse. In production, adapt the schema, persistence layer, and error handling to your environment before deploying.

Structured memory

```

session_state = {
    "profile_memory": {
        "user_role": "procurement_manager",
        "preferred_style": "concise"
    },

```

```

"task_state": {
    "objective": "summarize vendor risk",
    "constraints": ["cite sources", "use Q4 data"],
    "open_questions": []
},
"evidence_refs": [
    {"source_id": "risk_2025_q4", "type": "table"},
    {"source_id": "policy_irq_v3", "type": "document"}
]
}

```

Why it matters: This pattern reduces hidden waste by making decisions explicit, bounding context, or enabling safe reuse. In production, adapt the schema, persistence layer, and error handling to your environment before deploying.

Semantic cache

```

from hashlib import sha256

def make_cache_key(query_embedding, route, tenant_id, corpus_version, permission_scope):
    semantic_bucket = bucketize_embedding(query_embedding)
    raw = f"{semantic_bucket}|{route}|{tenant_id}|{corpus_version}|{permission_scope}"
    return sha256(raw.encode()).hexdigest()

def maybe_get_cached_answer(cache, key, freshness_tag):
    value = cache.get(key)
    if not value:
        return None
    if value["freshness_tag"] != freshness_tag:
        return None
    return value["answer"]

```

Why it matters: This pattern reduces hidden waste by making decisions explicit, bounding context, or enabling safe reuse. In production, adapt the schema, persistence layer, and error handling to your environment before deploying.

Confidence gate

```

def should_escalate(retrieval_score, rerank_margin, schema_valid, source_conflict,
                    high_stakes):
    low_retrieval = retrieval_score < 0.55
    weak_margin = rerank_margin < 0.08
    uncertain_structure = not schema_valid

    if high_stakes:
        return low_retrieval or weak_margin or uncertain_structure or source_conflict

    return (low_retrieval and weak_margin) or uncertain_structure or source_conflict

```


Why it matters: This pattern reduces hidden waste by making decisions explicit, bounding context, or enabling safe reuse. In production, adapt the schema, persistence layer, and error handling to your environment before deploying.

Token budget

```
MAX_PROMPT_TOKENS = {
    "cheap_direct": 1200,
    "rag_search": 2600,
    "reasoned_answer": 4000,
}

def enforce_budget(route, prompt_tokens):
    budget = MAX_PROMPT_TOKENS[route]
    if prompt_tokens > budget:
        raise ValueError(f"Prompt budget exceeded for {route}: {prompt_tokens} > {budget}")
```

Why it matters: This pattern reduces hidden waste by making decisions explicit, bounding context, or enabling safe reuse. In production, adapt the schema, persistence layer, and error handling to your environment before deploying.

Telemetry envelope

```
trace_payload = {
    "request_id": request_id,
    "route": route,
    "model": model_name,
    "prompt_tokens": prompt_tokens,
    "completion_tokens": completion_tokens,
    "retrieved_chunks": retrieved_count,
    "compressed_chunks": compressed_count,
    "cache_hit": cache_hit,
    "accepted": user_acceptance,
    "escalated": escalated,
}
emit_trace(trace_payload)
```

Why it matters: This pattern reduces hidden waste by making decisions explicit, bounding context, or enabling safe reuse. In production, adapt the schema, persistence layer, and error handling to your environment before deploying.

Quick reference: highest-impact interventions

Lever	Primary symptom	Typical savings logic	Main trade-off	Adoption priority
Reduce agent steps	Too many calls per request	Fewer model invocations and lower latency	Need clearer route design	Very high
Trim context	Prompt tokens climb every turn	Fewer tokens, clearer synthesis	Requires state design	Very high
Improve retrieval	Many chunks but weak grounding	Less noise, fewer retries	Needs corpus work	High
Route models by difficulty	Premium model share is high	Lower blended cost	Risk of misrouting	High
Layer caching	Repeated questions and repeated prompts	Reuse past work	Freshness and permission complexity	High
Precompute offline	Same reasoning repeated on stable data	Lower online cost and latency	Pipeline maintenance	Medium to high
Confidence escalation	Uncertain cases overconsume premium path	Narrow premium usage	Threshold tuning	Medium